

Feature Engineering

Onde o conhecimento de negócio vira número que o modelo consegue usar

Nenhum algoritmo, por mais sofisticado, recupera uma informação que nunca chegou até ele em forma numérica. Feature engineering é o processo de traduzir conhecimento de domínio — "cliente que compra toda sexta-feira", "sensor que oscila mais à noite" — em colunas que um modelo consegue processar. Este material cobre transformações numéricas, features temporais e de agregação, e as três famílias de método para decidir quais features vale manter.

Nível: Intermediário

Pre-requisitos: Análise Exploratória; Dados Faltantes e Outliers (módulos deste tema)

Carga sugerida: 8 a 10 horas (leitura + 3 notebooks)

Notebooks: 01-transformacoes-numericas · 02-features-temporais-e-agregacoes · 03-selecao-de-features · 99-exercicios

Autoria: Material do curso de ML & DL

Versao: 1.0

Sumário

1. Por que este módulo existe	3
1.1 O que você vai conseguir fazer ao final	3
2. Transformações numéricas	4
2.1 Log e potência: comprimindo a assimetria	4
2.2 Discretização (binning): quando faz sentido perder granularidade	4
2.3 Interações e razões: onde mora conhecimento de domínio	4
3. Features temporais: a categoria com mais armadilhas de vazamento . . .	6
3.1 Extraíndo estrutura de uma data	6
3.2 Features de defasagem (lag) e agregação móvel	6
3.3 Agregações por entidade (cliente, produto, loja)	6
4. Seleção de features: três famílias de método	8
5. Erros que custam caro — checklist	9
6. Para ir além	10

1. Por que este módulo existe

Modelos de gradient boosting e redes neurais são frequentemente descritos como "aprendem as features sozinhos" — e há verdade nisso, mas é uma meia-verdade perigosa. Uma árvore de decisão pode aprender que "compra às sextas" importa **se essa informação existir em alguma coluna**. Se o dataset só tem a data bruta de cada compra, a árvore jamais vai inventar sozinha o conceito de "dia da semana" — ela só enxerga o timestamp como um número gigante, quase sem padrão aproveitável. Feature engineering é o trabalho de expor, explicitamente, a estrutura que o modelo não tem como descobrir sozinho.

ANALOGIA

Dar dados brutos a um modelo é como entregar ingredientes crus a alguém que nunca cozinhou e pedir um prato pronto. Modelos modernos são bons cozinheiros — mas picar a cebola, temperar e pré-cozinhar o que precisa de pré-cozimento (as features) ainda é o que separa um prato mediano de um bom, mesmo com o melhor cozinheiro da casa.

1.1 O que você vai conseguir fazer ao final

- Escolher transformações numéricas (log, potência, binning) com um motivo técnico, não por costume.
- Construir features temporais e de agregação sem vazar informação do futuro para o passado.
- Aplicar os três tipos de seleção de features (filtro, wrapper, embutido) e saber quando cada um é apropriado.
- Reconhecer o custo (dimensionalidade, multicolinearidade, overfitting) de criar features demais.

2. Transformações numéricas

2.1 Log e potência: comprimindo a assimetria

Já visto no tema 1: dados de receita, tempo e contagem costumam ter cauda longa à direita. Uma transformação log comprime essa cauda e aproxima a distribuição de uma normal — o que ajuda modelos lineares (cuja função de perda assume, implicitamente, resíduos bem comportados) e métodos baseados em distância (onde um único valor extremo dominaria qualquer cálculo).

ARMADILHA COMUM

Log de zero é indefinido, e log de negativo não existe nos reais. A transformação padrão para lidar com zeros é $\log(1+x)$ (`np.log1p`), que preserva zero como zero. Para variáveis com valores negativos, é preciso um deslocamento antes, ou uma transformação de Box-Cox/Yeo-Johnson (que generaliza log e lida com negativos).

2.2 Discretização (binning): quando faz sentido perder granularidade

Transformar uma variável contínua em faixas (`idade` → "18-25", "26-35"...) parece um passo atrás, mas tem usos reais: captura relações **não-monotônicas** que um modelo linear não veria na variável contínua, aumenta a interpretabilidade para stakeholders de negócio, e reduz sensibilidade a outliers extremos dentro de cada faixa.

ARMADILHA COMUM

Discretizar sempre **perde informação** — duas pessoas de 26 e 34 anos, na mesma faixa "26-35", tornam-se indistinguíveis para o modelo. Para modelos que já capturam não-linearidade sozinhos (árvores, gradient boosting), binning manual raramente ajuda e frequentemente piora — a árvore já encontra o ponto de corte ótimo sozinha, com mais liberdade do que faixas fixas definidas a priori.

2.3 Interações e razões: onde mora conhecimento de domínio

Uma razão como `receita / n_funcionarios` (produtividade por funcionário) costuma carregar mais sinal que as duas variáveis originais separadas — porque codifica, de forma direta, o conceito de negócio que interessa. Interações (produto de duas features) permitem que um modelo linear capture efeitos que, sozinho, ele não conseguiria (uma reta não modela "o efeito de x_1 depende do valor de x_2 " sem um termo de interação explícito).

NO MERCADO

Em modelos lineares e regressão logística, interações e razões **precisam** ser criadas manualmente — o modelo não as descobre sozinho. Em árvores e gradient boosting, o modelo já captura muitas interações implicitamente (uma sequência de splits em x_1 e depois x_2 é uma forma de interação) — mas uma razão de negócio bem escolhida ainda ajuda, porque reduz a profundidade de árvore necessária para aprender o mesmo padrão.

ARMADILHA COMUM

Criar todas as interações possíveis entre p features gera $O(p^2)$ novas colunas — o mesmo problema de dimensionalidade e multicolinearidade visto no tema 2. `PolynomialFeatures(degree=2)` em um dataset com 50 colunas já produz mais de 1.200 features, a maioria ruído. Interações devem ser guiadas por hipótese de negócio ou filtradas depois, não geradas às cegas em massa.

3. Features temporais: a categoria com mais armadilhas de vazamento

3.1 Extraindo estrutura de uma data

Uma data bruta (2024-03-15 14:32:00) quase não tem valor preditivo direto — o valor está na estrutura que se pode extrair dela: dia da semana, hora do dia, é feriado, é fim de mês (comum em padrões financeiros), dias desde um evento de referência.

FORMULARIO

Encoding cíclico. Hora do dia e dia da semana são **circulares** — 23h e 0h são vizinhas, não extremos opostos. Codificar como um número linear (`hora = 23` vs. `hora = 0`) faz o modelo achar que estão longe. A correção padrão é o par seno/cosseno:

$$\sin\left(\frac{2\pi \cdot \text{hora}}{24}\right), \quad \cos\left(\frac{2\pi \cdot \text{hora}}{24}\right)$$

Esse par de coordenadas coloca cada hora num ponto de um círculo — 23h e 0h ficam geometricamente próximas, como deveriam.

3.2 Features de defasagem (lag) e agregação móvel

Prever vendas de amanhã a partir de vendas dos últimos 7 dias exige criar colunas como `vendas_ontem`, `media_movel_7_dias`, `vendas_mesma_semana_ano_passado`. Essas features são centrais em séries temporais (tema 7) e em qualquer problema onde o histórico de um cliente/entidade importa.

ARMADILHA COMUM

O vazamento temporal é o erro mais caro e mais comum desta seção. Calcular `media_movel_7_dias` usando uma janela que inclui o próprio dia que se quer prever (ou dias futuros) vaza informação do futuro. A regra não-negociável: toda feature de agregação temporal deve ser calculada usando **apenas dados estritamente anteriores** ao ponto que está sendo previsto. Em pandas, isso significa `shift(1)` antes de `rolling()`, sempre — nunca calcular a agregação e só depois alinhar.

3.3 Agregações por entidade (cliente, produto, loja)

`n_compras_ultimos_30_dias`, `ticket_medio_historico`, `dias_desde_ultima_compra` transformam um histórico de eventos em um snapshot por entidade — o padrão central de feature engineering em problemas de churn, crédito e recomendação.

ARMADILHA COMUM

A mesma armadilha de vazamento se aplica aqui: se o "histórico" usado para calcular `ticket_medio_historico` inclui a própria transação que está sendo classificada (ou transações futuras a ela), o modelo está usando informação que não existiria no momento real da decisão. Sempre pergunte: "no momento em que essa previsão seria feita de verdade, essa informação já existiria?"

4. Seleção de features: três famílias de método

Família	Como funciona	Vantagem	Limite
Filtro	Métrica univariada (correlação, informação mútua, variância) entre cada feature e o alvo, independente do modelo	Rápido, escalável	Ignora interações entre features
Wrapper	Testa subconjuntos de features treinando o modelo real (ex.: eliminação recursiva, RFE)	Considera o modelo final	Caro computacionalmente
Embutido	O próprio algoritmo de treino penaliza ou zera features (Lasso, importância de árvores)	Eficiente, informado pelo modelo	Específico do algoritmo usado

NOTA

Filtro por variância (remover colunas quase constantes) e por correlação entre pares de features (remover uma de cada par muito correlacionado) são passos baratos que valem rodar **sempre**, antes de qualquer método mais caro — eliminam candidatas óbvias e reduzem o custo dos métodos seguintes.

NO MERCADO

Lasso (regularização L1, tema 4) zera coeficientes de features irrelevantes automaticamente — é seleção de features embutida no próprio treino. Importância de features de Random Forest e gradient boosting (tema 4) cumpre papel semelhante, mas com uma ressalva séria: features de alta cardinalidade tendem a receber importância inflada artificialmente em árvores, mesmo sem relação real com o alvo (mais chances de splits "aleatoriamente úteis" apenas por terem mais valores possíveis).

5. Erros que custam caro — checklist

- Calcular uma feature de agregação temporal sem excluir explicitamente o próprio ponto (ou o futuro) da janela — vazamento silencioso.
- Codificar hora/dia da semana como número linear em vez de seno/cosseno, quebrando a proximidade circular.
- Gerar todas as interações possíveis (*PolynomialFeatures* em massa) sem filtro, multiplicando dimensionalidade e multicolinearidade.
- Discretizar variáveis que um modelo baseado em árvore já processaria melhor na forma contínua.
- Confiar em importância de features de árvore sem checar se ela não está inflada por cardinalidade alta.
- Criar razões de negócio sem checar divisão por zero ou por valores muito pequenos (que explodem a razão).

6. Para ir além

- Zheng & Casari, *Feature Engineering for Machine Learning* — cobertura prática e ampla, com o mesmo espírito deste módulo.
- Kuhn & Johnson, *Feature Engineering and Selection* — tratamento estatístico mais formal dos três tipos de seleção.
- Documentação do `sklearn.feature_selection` — implementações de referência para filtro, wrapper (RFE) e embutido.